Image Generation Evaluator

AI-Powered Image Generation,
Evaluation & Quality Scoring System

Project Overview:

The AI Image Generation & Evaluation Platform (AIGEP) is an advanced web-based system designed to generate high-quality synthetic images and rigorously evaluate their alignment, realism, and overall quality. Built on top of Stable Diffusion, CLIP, and an interactive Streamlit interface, the platform empowers developers, creative professionals, and researchers to produce and benchmark AI-generated imagery with precision.

AIGEP simplifies the entire image-generation workflow by intelligently interpreting user prompts, producing multiple visual outputs, and automatically assessing each image using deep-learning-based similarity metrics. By analysing semantic alignment with the input prompt, evaluating photographic realism, and computing final weighted scores, the system ensures transparent, repeatable, and statistically grounded image comparisons.

Through the integration of generative AI models, CLIP-based embedding analysis, and automated scoring and visualization tools, AIGEP provides a reliable environment for experimentation, model validation, creative exploration, and quality assurance in AI-driven image synthesis.

Scenario 1: Creative AI Development and Model Benchmarking

Developers, researchers, and machine learning practitioners working with generative AI models often struggle with inconsistent image quality, subjective evaluation methods, and the difficulty of comparing results across prompts, models, or configurations. The AI Image Generation & Evaluation Platform (AIGEP) solves this challenge by providing an end-to-end environment where users can generate images, measure semantic alignment, assess realism, and benchmark visual performance—objectively and at scale.

For example, a graphics research team experimenting with prompt engineering or comparing different Stable Diffusion checkpoints can use AIGEP to generate multiple variations of an image based on a single prompt. Each output is automatically evaluated using CLIP-based similarity scoring, realism descriptors, and weighted quality metrics. Instead of relying on manual inspection or subjective judgment, the team receives quantifiable, reproducible scores for each image, enabling clear insights into how model changes affect visual accuracy and quality.

With AIGEP's integrated visualization tools, users can compare images side-by-side, review alignment and realism scores, and analyze bar-chart summaries of model performance. This ensures that every generation cycle is not only creative but also measurable and statistically validated. Through its seamless integration of Stable Diffusion, CLIP embeddings, and Streamlit-based interactive controls, AIGEP accelerates experimentation, supports transparent

model evaluation, and empowers organizations to build more reliable, consistent, and high-quality generative image pipelines. Ultimately, AIGEP transforms subjective visual assessment into data-driven insight—driving progress in generative AI, creative tooling, and automated content quality assurance.

Scenario 2: Application Development, UI Testing, and Visual Quality Assurance

In modern software and application development, teams frequently require large volumes of diverse visual assets to validate user interfaces, stress-test rendering components, or train vision-based features. Relying on manually sourced images or copyrighted datasets can introduce compliance issues, licensing restrictions, and a lack of visual diversity. The AI Image Generation & Evaluation Platform (AIGEP) resolves this challenge by generating high-quality synthetic images that align with user-defined prompts while providing automated evaluation metrics to ensure consistency and accuracy.

For example, a mobile app development team building a photo-enhancement feature or an AI-powered filtering system can use AIGEP to generate hundreds of controlled, prompt-specific images representing various environments, lighting conditions, and object types. These generated images can then be evaluated for semantic accuracy and realism using the platform's CLIP-based scoring pipeline. Instead of relying on unpredictable real-world datasets, developers can rapidly produce targeted visual test cases that simulate production-grade scenarios.

AIGEP's integrated visualization tools and scoring charts allow teams to verify image quality, analyze alignment across prompts, and maintain consistent test datasets for regression testing. The system's ability to automatically re-generate new images based on the same prompt ensures reproducibility during development cycles. By providing privacy-safe generation, standardized evaluation, and automated scoring, AIGEP accelerates UI validation, strengthens product reliability, and reduces the effort required to maintain visual test assets. Ultimately, it empowers engineering teams to build, test, and refine applications faster—supported by a secure, scalable, and high-quality synthetic image pipeline.

Architecture Overview:

The AI Image Generation & Evaluation Platform (AIGEP) is a modular and intelligent system designed to generate high-quality synthetic images and score them based on semantic accuracy and visual realism. Developed using Streamlit as the interactive front-end interface, AIGEP integrates a seamless user experience with a powerful backend built on Stable Diffusion, CLIP, PyTorch, NumPy, and Pillow for advanced image generation, embedding extraction, and scoring workflows.

The platform follows a layered architecture consisting of three core backend modules: ImageGenerator, ImageEvaluator, and VisualizationEngine, each fulfilling a specialized function in the pipeline.

- The ImageGenerator module leverages Stable Diffusion to generate multiple image variations based on user prompts, model settings, sampling parameters, and seeds.

- The ImageEvaluator module computes prompt-image alignment using CLIP embeddings, realism scores using descriptive text comparisons, and final weighted scores that combine semantic accuracy with visual quality.

- The VisualizationEngine produces comparative bar charts, score summaries, and thumbnail galleries that enable transparent benchmarking and easy interpretation of results.

These backend components are unified through a Streamlit-powered front-end, offering prompt input, generation controls, evaluation settings, and interactive visualization tabs. Users can generate images, evaluate them, inspect structured JSON results, and review statistical summaries—all within a single, responsive UI. With built-in scoring, reproducible evaluation, and model-agnostic generation flexibility, AIGEP provides a complete ecosystem for creative experimentation, research testing, and quality assurance in generative image workflows.

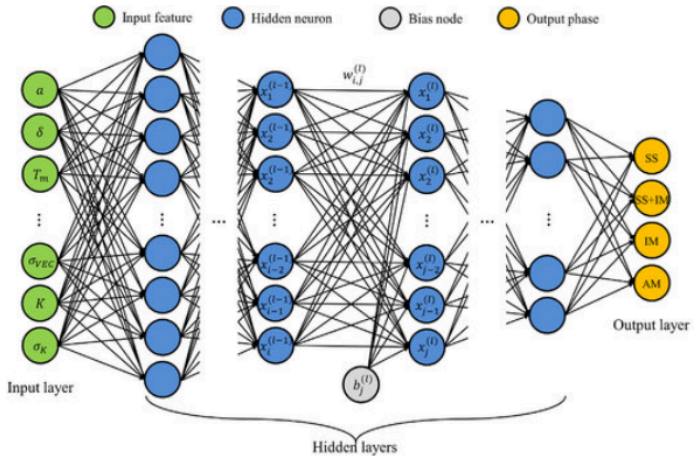

Figure 1: AI/ML System Architecture Illustration

Core Technologies:

The AI Image Generation & Evaluation Platform (AIGEP) is built using a robust stack of modern machine learning, visualization, and web technologies. Each component plays a critical role in enabling prompt-based image generation, semantic evaluation, and interactive user experience.

• Streamlit (Frontend Framework)

Provides an intuitive, real-time, and interactive interface for writing prompts, adjusting generation settings, triggering evaluations, and visualizing results. Streamlit handles page layout, tab navigation, image previews, and result dashboards.

• Stable Diffusion (Image Generation Engine)

Forms the core generative model responsible for creating high-quality images from textual prompts. It supports seed control, adjustable guidance scales, and fine-tunable inference steps for customizable output.

• CLIP (Semantic Evaluation Model)

Used to compute alignment between text prompts and generated images. CLIP converts both into vector embeddings and measures similarity to score semantic correctness and coherence.

• PyTorch (ML Framework)

Serves as the foundational deep learning engine powering both Stable Diffusion and CLIP. It ensures fast tensor computations, GPU acceleration, and reliable model execution.

• NumPy & Pillow (Image Processing Utilities)

NumPy handles computations such as vector operations, score normalization, and statistical measurements. Pillow manages image loading, processing, saving, and format handling during the pipeline.

• Matplotlib (Visualization Framework)

Generates score distribution charts, bar graphs, and comparison plots that help users understand alignment, realism, and final score metrics in an interpretable format.

• Backend Logic (Core Processing Layer)

Orchestrates all interactions between the UI, generation model, evaluation modules, and visualization engine — ensuring modularity, scalability, and high-performance execution.

Component-Wise Architecture:

Component	Description
User Interface (Streamlit)	Interactive dashboard for entering prompts, selecting generation/evaluation parameters, and displaying generated images, scores, and visualizations. Handles session state and real-time updates.
Stable Diffusion Pipeline Loader (`load_sd_pipeline`)	Loads and initializes the Stable Diffusion model with optional scheduler and attention slicing. Ensures efficient GPU/CPU usage and supports multiple inference steps and guidance scale.
Image Generator (`generate_images`)	Generates images from user prompts using the loaded pipeline. Supports multiple samples, seed-based reproducibility, guidance scaling, and step configuration. Saves images to the output directory.
CLIP Loader (`load_clip`)	Loads CLIP model and processor for semantic evaluation of images against text prompts. Supports embedding computation for text and images.
Image Encoder (`encode_image`)	Computes normalized CLIP embeddings for generated images to allow semantic similarity comparison.
Text Encoder (`encode_text`)	Computes normalized CLIP embeddings for textual prompts to allow similarity evaluation with images.

Evaluator (`evaluate_images`)	Calculates alignment scores (prompt-image similarity), optional realism scores (via descriptors), and final weighted scores for all images in the output directory. Supports auto-generation if no images exist.
Realism Scorer (`realism_score_via_descriptors`)	Computes realism score of images by comparing CLIP embeddings with predefined realism descriptors like “a photograph” or “a realistic photo.”
Cosine Similarity Calculator (`cosine_sim`)	Computes similarity between embeddings (image vs text) to generate alignment metrics.
Visualization Engine (`plot_scores`)	Generates bar charts of final image scores for quick visual comparison and saves output charts.
Backend Logic Controller (`final_score`)	Combines alignment and realism scores into a final score using configurable weighting. Orchestrates the evaluation workflow.
Session State Management	Keeps track of generated images and evaluation results within the Streamlit session to allow persistent UI display.
Helper Utilities (`ensure_dir`, `save_json`, `auto_device`)	Utility functions for directory creation, JSON saving, and automatic device (CPU/GPU) selection.

Image Generation Evaluator

Pre-requisites:

Before using the AI Image Generation & Evaluation Platform (AIGEP), ensure that your development environment is properly configured. The following components and tools are required:

1. Python Environment Setup

AIGEP is built using Python 3.10+, leveraging PyTorch, NumPy, and other libraries for image generation, embedding extraction, and scoring.

- Create and activate a dedicated virtual environment (e.g., `aigep_env`) to manage dependencies and maintain a clean setup.

- Official Python documentation: https://www.python.org/downloads/
  Suggested image: Python 3 logo or virtual environment illustration.

2. Streamlit Installation & Configuration

Streamlit powers the interactive front-end dashboard where users input prompts, control generation parameters, and view evaluation results.

- Install Streamlit via pip: `pip install streamlit`

- Streamlit documentation: https://docs.streamlit.io/library/get-started/installation

- Tutorial: https://www.youtube.com/watch?v=JwSS70SZdyM
  Suggested image: Streamlit logo or dashboard mockup.

3. Library Installation & Core Dependencies

AIGEP relies on several Python libraries for image generation, evaluation, and visualization. Install all dependencies using `requirements.txt`.

Key libraries include:

- streamlit – Interactive UI and image display

- diffusers & torch – Stable Diffusion generation

- transformers – CLIP model for embedding and scoring

- numpy – Numerical computation
- Pillow – Image handling
- matplotlib – Score visualization
  Suggested image: Logos of key libraries (NumPy, PyTorch, Pillow, Matplotlib).

4. GPU / PyTorch Setup (Optional but Recommended)

To accelerate image generation:

- Ensure a compatible CUDA-enabled GPU is installed.
- PyTorch installation guide: https://pytorch.org/get-started/locally/
  Suggested image: GPU illustration or PyTorch CUDA logo.

5. Development Environment

Recommended IDEs for efficient development and debugging:

- Visual Studio Code: https://code.visualstudio.com/
- PyCharm (Community Edition): https://www.jetbrains.com/pycharm/download/
  Suggested image: VS Code or PyCharm interface screenshot.

6. Optional Learning Resources

- Streamlit Components Gallery: https://streamlit.io/components
- Stable Diffusion / Diffusers Docs: https://huggingface.co/docs/diffusers/index
- CLIP Model Documentation: https://huggingface.co/docs/transformers/model_doc/clip
  Suggested image: Streamlit or Hugging Face documentation screenshot.

Project Flow:

1. **Environment Setup and Dependency Configuration**
   • **Activity 1.1:** Create and activate a Python virtual environment (e.g., `aigep_env`) and install required dependencies (`torch`, `diffusers`, `transformers`, `streamlit`,

`numpy`, `Pillow`, `matplotlib`, etc.).
• **Activity 1.2:** Organize project structure with separate modules for generation, evaluation, visualization, and UI styling (e.g., `styles.css`).
• **Activity 1.3:** Initialize Streamlit app and validate integration between front-end and backend modules.

2. **Core Logic Development (Image Generation & Evaluation Engine)**
   • **Activity 2.1:** Implement ImageGenerator module to produce images from textual prompts using Stable Diffusion, with seed control, guidance scale, and step configuration.
   • **Activity 2.2:** Develop ImageEvaluator module to compute CLIP-based semantic alignment, realism scores, and final weighted scores.
   • **Activity 2.3:** Integrate backend logic to manage evaluation, reproducibility, and optional storage of generated images and scores.

3. **Streamlit UI Implementation and User Interaction**
   • **Activity 3.1:** Design a multi-tab layout with navigation, real-time updates, and professional styling.
   • **Activity 3.2:** Configure user input system for prompts, number of images, steps, guidance scale, and session state.
   • **Activity 3.3:** Display generated images, computed metrics, evaluation results, and visualizations interactively.

4. **Testing, Optimization, and Deployment**
   • **Activity 4.1:** Test UI components and verify accurate image generation.
   • **Activity 4.2:** Validate evaluation module and ensure alignment and realism metrics are consistent.
   • **Activity 4.3:** Optimize pipeline performance, conduct end-to-end testing, and prepare deployment for research or production use.

MILESTONE 1: Environment Setup and Dependency Configuration

This foundational milestone establishes the technical environment required for building and deploying the **AI Image Generation & Evaluation Platform (AIGEP)**. It ensures that all dependencies, frameworks, and integrations are configured correctly for seamless execution of image generation, evaluation, and visualization workflows.

Activity 1.1: Python Environment and Dependency Installation
• Create and activate a dedicated virtual environment for AIGEP to maintain dependency isolation.

```
SELECT COMMAND PROMPT IT WILL NOT RUN IN POWERSHELL
-->python -m venv myenv

for certain versions
-->py -3.10 -m venv myenv

Once the environment is created, activate it by running
-->.\myenv\Scripts\activate

With the environment activated you can install packages using pip
-->pip install package_name
```

Figure 2: *Creating & Activating Environment*

• Install all required dependencies listed in `requirements.txt` using:
`torch`, `diffusers`, `transformers`, `streamlit`, `numpy`, `Pillow`, `matplotlib`

• Verify compatibility with **Python 3.10+** and confirm successful installation of all modules.

• Test library imports and ensure ML, image, and visualization packages (`torch`, `diffusers`, `Pillow`, `matplotlib`) work correctly.

Activity 1.2: Project Structure Initialization
• Organize project modules into a structured functions single file code hierarchy for clarity and maintainability:

- `streamlit` – Front-end interface and main application logic
- `generate_images` – Image generation functions using Stable Diffusion
- `evaluate_images` – Evaluation, scoring, and realism calculations

Activity 1.3: Streamlit Application Initialization
• Run `streamlit run main.py` in terminal to initialize

• Streamlit ensures consistent theming, responsive layout, and rendering of all components across devices.

• Validate integration between all modules, confirming smooth data flow between the UI, image generation pipeline, evaluation engine, and visualization outputs.

MILESTONE 2: Core Logic Development (Generation & Evaluation Engine)

This milestone focuses on building the core functional engine of AIGEP, responsible for generating high-quality images using Stable Diffusion and evaluating them using CLIP-based semantic and realism scoring mechanisms. It establishes the essential backend logic required for intelligent and automated image generation workflows.

Activity 2.1: Image Generation Module Development

• Implement the Stable Diffusion pipeline loader (`load_sd_pipeline`) to manage model loading, scheduler configuration, precision settings, and device optimization.
• Develop the `generate_images()` function to create multiple image samples from user prompts with support for seed control, inference steps, and guidance scale.
• Save generated outputs in a structured directory format for further evaluation and UI display.

```
model_name = model_name or os.getenv("SD_MODEL", "runwayml/stable-diffusion-v1-5")
device = auto_device(device or os.getenv("DEVICE"))
pipe = load_sd_pipeline(model_name, device=device)
ensure_dir(out_dir)
```

Figure 3: Model Configuration

Activity 2.2: Evaluation and Scoring Engine Implementation

• Integrate CLIP model loading (`load_clip`) for text–image semantic comparison.
• Build the `encode_text()` and `encode_image()` functions to compute normalized embeddings for scoring.
• Implement the evaluation workflow via `evaluate_images()` to calculate alignment scores, realism scores, and final weighted metrics.
• Add realism enhancement using descriptor-based scoring (e.g., “a realistic photo”).

```
def load_clip(model_name: str = "openai/clip-vit-base-patch32", device: str = "cpu"):
    model = CLIPModel.from_pretrained(model_name).to(device)
    processor = CLIPProcessor.from_pretrained(model_name)
    return model, processor
```

```
def encode_text(model: CLIPModel, processor: CLIPProcessor, text: str, device: str = "cpu"):
    inputs = processor(text=[text], return_tensors="pt", padding=True).to(device)
    with torch.no_grad():
        emb = model.get_text_features(**inputs)
    emb = emb / emb.norm(p=2, dim=-1, keepdim=True)
    return emb.cpu().numpy()[0]
```

Figure 4 & 5: Model Configuration

Activity 2.3: Backend Integration and Workflow Management

• Connect all core modules to ensure smooth execution from prompt input → generation →

scoring → visualization.
• Implement helper utilities (`ensure_dir`, `auto_device`, `save_json`) to maintain clean workflow management.
• Validate pipeline performance and correctness using multiple prompts and parameter settings.

```
if st.button("Evaluate Images"):
    try:
        with st.spinner("Evaluating images..."):
            device = None if device_option == "auto" else device_option
            results = evaluate_images(outputs_dir, prompt,
                                      clip_model_name=clip_model,
                                      device=device,
                                      compute_realism=compute_realism,
                                      alignment_weight=float(alignment_weight),
                                      auto_generate_if_empty=auto_generate_if_empty,
                                      gen_n_samples=int(n_samples),
                                      gen_seed=(None if int(seed) == 0 else int(seed)),
                                      gen_guidance=float(guidance),
                                      gen_steps=int(steps),
                                      sd_model_name=sd_model)
            st.session_state.evaluation_results = results
            save_json(results, Path(outputs_dir) / "evaluation_results.json")
            chart_path = plot_scores(results, out_path=Path(outputs_dir) / "scores_bar.png")
        st.success("Evaluation complete.")
    except Exception as e:
        st.exception(e)
```

Figure 6: Model Configuration

MILESTONE 3: Streamlit UI Implementation & User Interaction

This milestone establishes the interactive front-end layer of AIGEP, enabling users to generate, analyze, and visualize images seamlessly through a responsive and intuitive Streamlit dashboard.

Activity 3.1: User Interface and Multi-Tab Layout Design
• Create a structured multi-tab UI to separate generation, evaluation, and visualization workflows.
• Apply professional styling using `streamlit` for consistent layout, colors, spacing, and responsiveness.
• Ensure mobile-friendly design and component alignment across screens.

```
def run_streamlit_ui():
    st.set_page_config(layout="wide", page_title="Image Generation Evaluator")
    st.title("Image Generation Evaluator")
```

```
with st.sidebar:
    st.header("Generation / Evaluation settings")
    sd_model = st.text_input("Stable Diffusion model", value=os.getenv("SD_MODEL", "runwayml/stable-diffusion-v1-5"))
    clip_model = st.text_input("CLIP model", value=os.getenv("CLIP_MODEL", "openai/clip-vit-base-patch32"))
    device_option = st.selectbox("Device", ["auto","cuda","cpu"], index=0)
    outputs_dir = st.text_input("Outputs directory", value="outputs")
    ensure_dir(outputs_dir)
```

```
if st.button("Generate Images"):
    s = None if int(seed) == 0 else int(seed)
    try:
        with st.spinner("Generating images (this may take a while on first run)..."):
            device = None if device_option == "auto" else device_option
            imgs = generate_images(prompt, out_dir=outputs_dir, n_samples=int(n_samples), seed=s,
                                   guidance_scale=float(guidance), steps=int(steps),
                                   model_name=sd_model, device=device)
            st.session_state.generated_images = imgs
        st.success(f"Generated {len(imgs)} images.")
    except Exception as e:
        st.exception(e)
```

Figure 7, 8 & 9: Streamlit Graphical User Interface

Activity 3.2: Input Configuration and Session State Management

• Add interactive inputs for entering prompts, number of images, steps, seed, and guidance scale.

• Implement Streamlit session state to persist generated images, evaluation results, and charts during navigation.

• Validate user inputs and apply error handling for invalid or missing fields.

Activity 3.3: Output Rendering and Data Visualization

• Display generated images directly within the Streamlit interface with numbering and captions.

• Render evaluation metrics, semantic alignment values, realism scores, and final weighted results.

• Integrate visual score charts generated using Matplotlib for easy comparison of image quality.

• Provide JSON output of evaluation results for export or further analysis.

```
if st.session_state.evaluation_results:
    top = st.session_state.evaluation_results[:8]
    cols = st.columns(min(4, len(top)))
    for i, r in enumerate(top):
        with cols[i % 4]:
            realism_text = "NA" if r["realism_score"] is None else f"{r['realism_score']:.3f}"
            caption = f"Final: {r['final_score']:.3f} | Align: {r['alignment_score']:.3f} | Realism: {realism_text}"
            st.image(r["image"], caption=caption, use_column_width=True)
    st.subheader("Full JSON results")
    st.json(st.session_state.evaluation_results)
    scores_png = Path(outputs_dir) / "scores_bar.png"
    if scores_png.exists():
        st.image(str(scores_png), caption="Score Visualization")
```

```
    st.session_state.evaluation_results = results
    save_json(results, Path(outputs_dir) / "evaluation_results.json")
    chart_path = plot_scores(results, out_path=Path(outputs_dir) / "scores_bar.png")
st.success("Evaluation complete.")
```

Figure 10 & 11: Model Configuration

MILESTONE 4: Testing, Optimization, and Deployment

This milestone ensures the platform meets performance, accuracy, and reliability standards through structured testing and optimization. It prepares AIGEP for real-world deployment and end-to-end validation.

Activity 4.1: Functional Testing and Module Verification

- Test all UI components, form inputs, generation parameters, and scoring outputs for correctness.
- Validate Stable Diffusion model behavior across various prompt types and parameter settings.
- Ensure CLIP scoring remains consistent and reproducible under seed-controlled conditions.

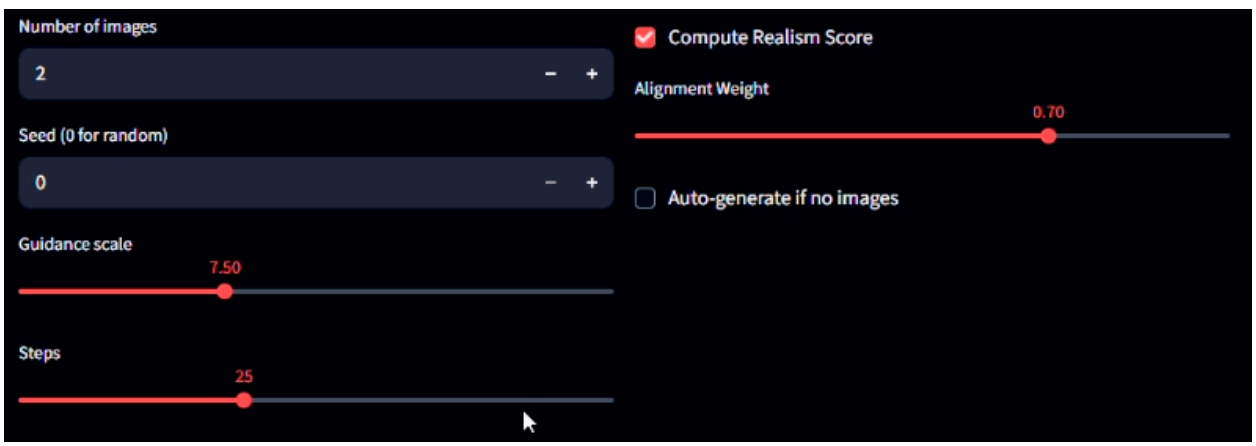

Figure 12: UI Components Visual

Activity 4.2: Performance Testing and Optimization

- Measure runtime performance for image generation and scoring under different batch sizes.
- Optimize GPU/CPU usage using attention slicing, half-precision, and efficient scheduler settings.
- Test system performance with multiple images and large prompts to ensure reliability.

Figure 13: UI Components Visual

Activity 4.3: Deployment Preparation and Final Validation

• Prepare the application for deployment (Streamlit Cloud, local server).

• Conduct final end-to-end validation covering prompt input → image generation → scoring → visualization.

• Verify folder structure, logging, error handling, and result storage before release.

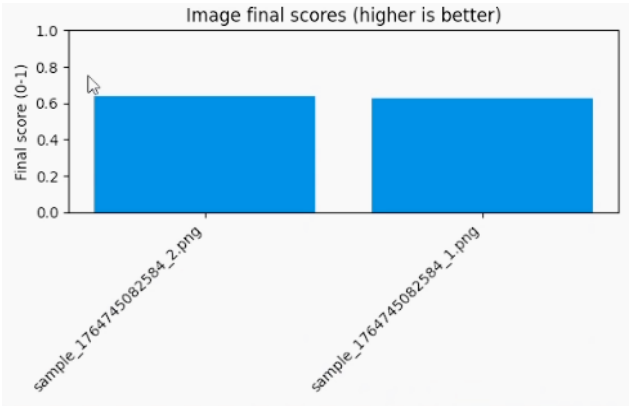

Figure 14, 15 & 16: Result UI Visual Scores JSON and BAR Chart

Conclusion:

The AI Image Generation & Evaluation Platform (AIGEP) represents a significant advancement in intelligent content creation, transforming how developers, researchers, and creative teams generate, assess, and refine synthetic images. By integrating state-of-the-art image generation models with robust CLIP-based evaluation and visualization techniques, the system ensures that every generated output is meaningful, high-quality, and aligned with the user's intent. Whether producing multiple images from a single prompt, analyzing semantic consistency, or measuring realism, AIGEP empowers users to create and validate AI-generated images with exceptional precision and efficiency.

Built on a streamlined Streamlit interface, the platform delivers a seamless, interactive user experience. Users can easily configure prompts, control generation parameters, visualize outputs, compare semantic scores, and interpret evaluation metrics—all within a clean and intuitive dashboard. The combination of Stable Diffusion, CLIP embeddings, optimized backend processing, and visual scoring charts makes AIGEP a powerful and practical tool suitable for real-world workflows, including research, experimentation, benchmarking, and creative exploration.

Looking ahead, AIGEP holds strong potential for further expansion. Enhancements such as multi-model support, advanced realism scoring mechanisms. Ultimately, this project demonstrates how intelligent generation and automated evaluation can accelerate creative development cycles, strengthen model reliability, and empower users to explore the possibilities of AI-driven image synthesis with confidence and clarity.